PIOE — Programmable Protocol Converter Chip

Specification v0.1

Target: Tiny Tapeout shuttle submission. **Status:** Draft — spec-first, RTL not yet started. **Date:** 12 October 2026.

1. Project Goal

A small, standalone, firmware-programmable chip that converts between UART, SPI, and I2C — any protocol to any other protocol, in either direction (UART↔SPI, UART↔I2C, SPI↔I2C), not a fixed single-direction bridge.

The chip has no CPU, no OS, and no permanent connection to a host computer. It is programmed once via an external SPI flash chip, then runs standalone, power-cycle to power-cycle, like a microcontroller that boots from its own flash — except the "flash" here is an external chip, since Tiny Tapeout's standard digital flow has no on-die flash module available.

Explicitly out of scope / deferred:

- On-chip flash (not offered by the SKY130 / IHP standard-cell flows Tiny Tapeout uses)
- Live reprogramming without a power cycle (rejected in favor of simplicity — see Section 4)
- Full packet buffering in on-chip memory (no room for it — see Section 5)

2. Origin / Alternatives Considered

This design was chosen after evaluating other Tiny Tapeout project ideas.

- **SD card reader → UART bridge**: feasible, but judged the weakest resume/learning story of the options considered. Kept as an FPGA warm-up exercise, not the tapeout target.
- **Bit-serial RISC-V core (SERV-style)**: technically impressive, but already well-represented on past Tiny Tapeout shuttles, with risk of looking derivative. A similar project ("ProtoCore," a firmware-programmable serial protocol engine with SPI program loading, built for a Jane Street/Tiny Tapeout IHP challenge) was found during research — worth reviewing later so this design doesn't duplicate it, though it doesn't block this effort.

- **Other "think outside the box" alternatives considered**: a hardware IMU-based failsafe "reflex arc" for drones, a silicon-characterization chip (PUF/TRNG/ring-oscillator sensing), and an ASCON crypto accelerator. Not pursued for this slot.

The chosen direction — a reprogrammable protocol engine inspired by the RP2040's PIO peripheral and the bit-serial "trade time for area" principle — was picked as the most practical and broadly reusable option.

3. Pin Budget

Tiny Tapeout provides 24 digital I/O pins: 8 dedicated input, 8 dedicated output, 8 bidirectional — plus separate `clk` and `rst_n` pins, not counted against the 24.

Function	Pins	Notes
Boot SPI link (to external flash)	4	CS, SCK, MOSI, MISO — dedicated, never repurposed
Chip enable (active-low)	1	See Section 4
General-purpose protocol pins	~17 remaining	Shared across UART / SPI / I2C generation at runtime, via an internal mux the loaded program controls

Design decision: the boot-flash SPI link is physically separate from the general-purpose runtime pins. Reusing pins between boot and run phases was considered (saves pins) but rejected for simplicity, to avoid a mode-dependent rewiring problem on the same pads across two very different operating phases.

At runtime, the general-purpose pins are shared across target protocols — e.g., the same physical pins act as SPI SCK/MOSI/MISO/CS at one moment and I2C SDA/SCL at another — controlled by an output mux driven by the currently loaded program. This is safe because the chip only runs one loaded program at a time; SPI and I2C are never active simultaneously on the same pins.

4. Boot / Programming Flow

1. **Programming (offline):** the user writes a program into an external SPI flash chip from outside the system — e.g., from a PC — independent of the PIOE chip itself.
2. **Connection:** the programmed flash chip is wired to the PIOE chip's dedicated boot-SPI pins.

3. **Enable:** an active-low enable pin gates operation. Pin low → chip boots: reads the fixed-length program from external flash into its internal instruction slots over the dedicated SPI link, then begins executing from slot 0. Pin high (or floating via pull-up) → chip held idle / in reset, minimal power draw. Active-low was chosen so a disconnected or default-state pin draws minimal power rather than needing an active drive just to stay idle.
4. **Reprogramming:** to change behavior, the user writes new contents to the external flash chip, then power-cycles (toggles enable) to reload. There is no live, mid-run reprogramming path — a deliberate simplification.
5. **No corruption handling, no fallback mode:** if flash is blank or garbled, the chip loads and executes whatever is there. No boot-mode pin, no safe/recovery mode. Accepted limitation for v1.

Analogy used during design discussion: this mirrors a microcontroller's boot-from-flash, reset-vector behavior, with one key difference. A real MCU typically executes continuously out of its flash; this chip, having only a small fixed number of instruction slots (currently planned around 16), copies the full program into internal slots once at boot and then runs entirely from internal state. Flash here is boot-time storage only, not execution-time storage.

5. Memory Constraints

Why this architecture looks the way it does: Tiny Tapeout's standard-cell digital flow has no practical on-chip memory for anything resembling a buffer — a 512-byte SD sector alone would need roughly 4,000 flip-flops, far beyond a tile's budget. Two consequences shaped this spec:

- **No on-chip flash.** Not just a density problem — the fabrication process itself doesn't offer flash cells on the shuttles Tiny Tapeout uses. Hence the external-flash-plus-bootloader workaround in Section 4.
- **No packet buffering.** "Store-and-forward" conversion can't mean literally storing a packet in registers. Instead, conversion is streamed byte-by-byte: a byte is pulled in, shifted out on the target protocol, and a result byte is pushed back out, with the "transaction" existing as a flow through the engine rather than as stored state.
-

6. The I2C Open-Drain Problem

This was the main electrical issue worked through during spec development, and is worth documenting since it drove a real hardware decision.

The problem

I2C's SDA and SCL lines are shared by multiple devices on the same bus. If any device could actively drive the line high, two devices could contend — one driving high, another driving low — causing a direct short between drivers (signal corruption at best, pin damage at worst).

I2C avoids this by convention: no device may ever actively drive the bus high. Each device may only pull the line low (actively drive low), or release the line (high-impedance / disconnect). When all devices release, an external pull-up resistor passively restores the line to high. This drive scheme is called open-drain — categorically different from SPI, where every signal is conventional push-pull, always actively driven high or low by exactly one side.

Why this matters for PIOE

The engine's output pins, as designed for SPI/UART-style bit-banging, are ordinary push-pull: a logical 1 in the program actively drives the pin high. That is exactly the behavior I2C prohibits. This applies any time I2C is active on either side of a conversion — it is not specific to any particular protocol pairing.

Options considered

1. **Software discipline only**: keep pins simple push-pull, add external pull-ups, and rely on the loaded program never issuing a "drive high" on an I2C pin. Simplest silicon, but puts a bus-fighting failure mode one bug away, in every future program anyone writes for this chip (including other users, since this chip may see use beyond its original author).
2. **Hardware-enforced open-drain emulation (chosen)**: add a small amount of mode-controlled logic between the engine's output-data bit and the pin's existing output-enable control. Standard-cell I/O pads already provide an independent output-enable signal separate from output-data, so this doesn't require new pad hardware — just logic to interpret it differently in I2C mode. Program wants to output 0 → drive low (output-enable on, output-data 0). Program wants to output 1 → don't drive (output-enable off); external pull-up restores high. Estimated cost: on the order of a few gates per I2C-capable pin (e.g., a single AND/mux stage) — small relative to the safety gained.

Decision: build real open-drain support into hardware, selectable per-mode (push-pull for SPI, open-drain for I2C) via the pin mux, rather than relying on program correctness alone.

7. Master / Slave Capability

Goal (provisional): the chip should be able to act as both master and slave, on both SPI and I2C sides of a conversion — e.g., relaying data in from an external microcontroller acting as SPI master, while PIOE itself acts as I2C master driving a sensor on the other side, or any other combination.

This is held as the target but explicitly not a hard requirement: if full master/slave generality on both protocols proves too complex to implement reliably within the chip's constraints, it will be scaled back to master-only. This will be assessed once the individual protocol blocks are built and their real complexity is visible (see Section 8).

8. Build Order

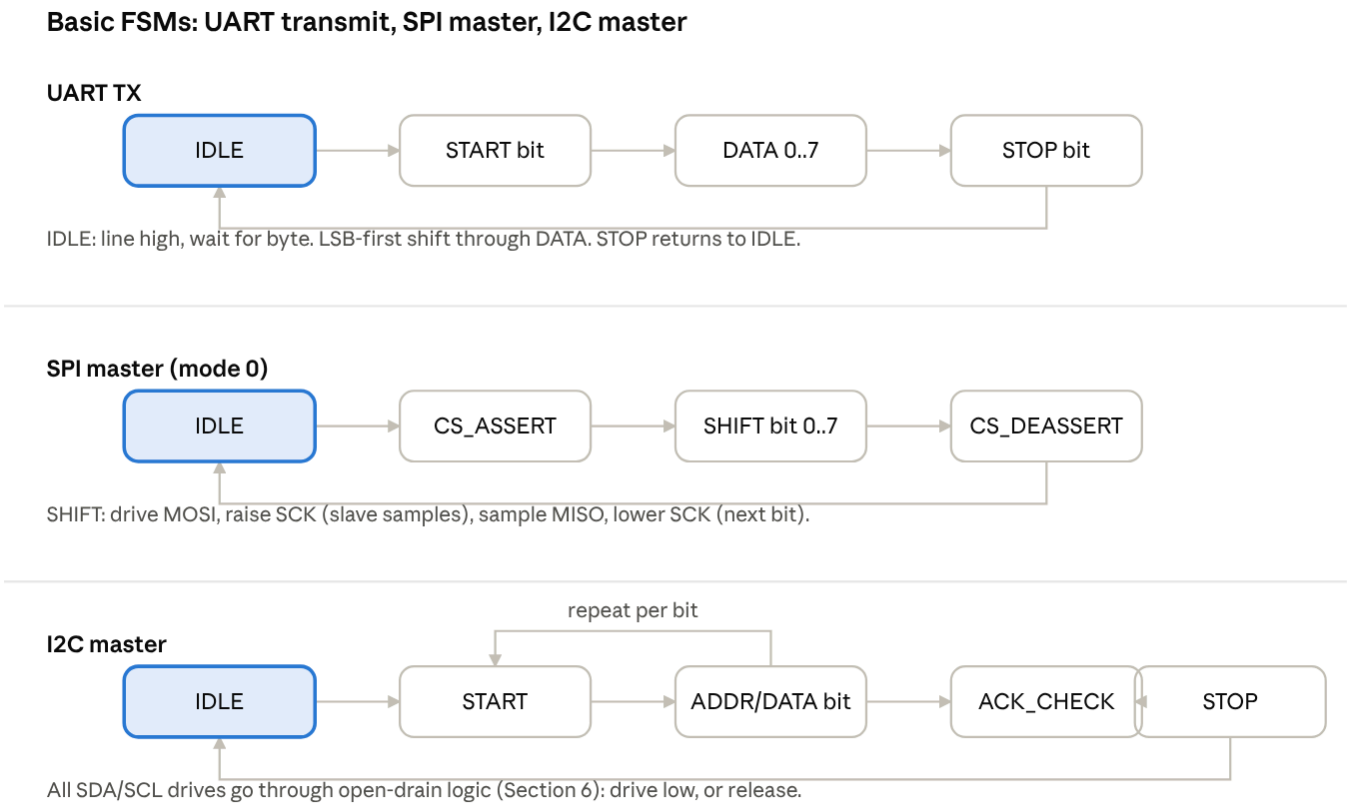
Agreed approach: bottom-up, not top-down.

1. Build each protocol block (UART, SPI, I2C) independently first.
2. Check gate/transistor count for each block in isolation.
3. Verify functionality of each block on its own (simulation, then FPGA bring-up).
4. Only then design the control/sequencing logic that ties blocks together for conversion between them.

This order was chosen deliberately so that ISA details (e.g., whether I2C needs dedicated opcodes like `START` / `STOP` / `ACK`, versus handling it through generic primitives) are decided from real block behavior rather than guessed up front.

9. Basic FSM Sketches

Starting points for the individual block designs in step 8.1, not final RTL.



basic FSMs — UART transmit, SPI master, I2C master

UART RX (not drawn) mirrors TX: wait for a falling edge (start bit) from IDLE, sample each of 8 data bits at mid-bit, check the stop bit, return to IDLE. SPI slave mode would react to an externally driven CS/SCK rather than generating them — deferred until the master-only block works (Section 7).

10. Open Items / Not Yet Decided

- Final ISA opcode list (deferred until after block-level FSMs are implemented — see Section 8)
- Engine clock source and prescaler values (depends on target UART baud rate and I2C/SPI bus speeds — not yet specified)
- Whether I2C gets dedicated opcodes (`START`, `STOP`, `ACK`) or is built from generic primitives — deferred per Section 8
- Clock-stretching support for I2C (slave holding SCL low to pause the master) — not yet addressed
- Exact instruction count ceiling (v0 prototype used 16 slots; may change based on real block sizes)
- Final master/slave scope per Section 7